TDD for Beginners

Step-by-Step Guide

Project: Myngly Ads Unified API **Date:** 2026-01-28 **Audience:** Developers new to Test-Driven Development

---

What is TDD?

TDD (Test-Driven Development) means **writing tests BEFORE writing code**.

It follows 3 simple steps:

Step	Name	What to do
1	**RED**	Write a failing test
2	**GREEN**	Write minimal code to pass the test
3	**REFACTOR**	Clean up the code (tests still pass)

Why TDD?

- Catch bugs early
- Code is always tested
- Forces you to think about requirements first
- Gives confidence when refactoring
- Documentation through tests

Step 1: Setup Your Environment

Open a terminal in the project folder and run:

```
cd C:/Users/Usuario/Myngly_ads_unified_api_1_27

# Install dependencies (if not done)
npm install

# Verify tests work
npm test
```

You should see: **1,008 passed**

Step 2: Understand the Test Structure

Tests live next to the code they test in a `__tests__` folder:

```
src/domains/identity/services/
├── auth.service.ts          ← The actual code
└── __tests__/
    └── auth.service.test.ts ← Tests for that code
```

File Naming Convention

Code File	Test File
`auth.service.ts`	`auth.service.test.ts`
`user.repository.ts`	`user.repository.test.ts`
`email.utils.ts`	`email.utils.test.ts`

Step 3: Write Your First Test (RED)

Let's say you want to add a new function `isEmailValid()` to check emails.

Create the test file FIRST (before writing the function):

```
// src/shared/utils/__tests__/email.utils.test.ts

import { describe, it, expect } from 'vitest';
import { isEmailValid } from '../email.utils';

describe('isEmailValid', () => {
  it('should return true for valid email', () => {
    expect(isEmailValid('user@example.com')).toBe(true);
  });

  it('should return false for invalid email', () => {
    expect(isEmailValid('not-an-email')).toBe(false);
  });

  it('should return false for empty string', () => {
    expect(isEmailValid('')).toBe(false);
  });
});
```

Run the test:

```
npm test -- email.utils.test.ts
```

Result: FAILS (RED)

The function doesn't exist yet - this is expected!

Step 4: Write Minimal Code (GREEN)

Now create the function to make tests pass:

```
// src/shared/utils/email.utils.ts
```

```
export function isEmailValid(email: string): boolean {
  if (!email) return false;
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}
```

Run the test again:

```
npm test -- email.utils.test.ts
```

Result: PASSES (GREEN)

All 3 tests pass!

Step 5: Refactor (IMPROVE)

Now you can improve the code while keeping tests green:

```
// Improved version with better regex and validation
const EMAIL_REGEX = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;

export function isEmailValid(email: string): boolean {
  if (!email || typeof email !== 'string') return false;
  return EMAIL_REGEX.test(email.trim());
}
```

Run tests again

They should still pass! If not, you broke something - fix it.

Step 6: Add More Test Cases

Think of edge cases and add tests:

```
describe('isEmailValid', () => {
  // ... existing tests ...

  it('should return false for email without domain', () => {
    expect(isEmailValid('user@')).toBe(false);
  });

  it('should handle email with spaces', () => {
    expect(isEmailValid('  user@example.com  ')).toBe(true);
  });

  it('should return false for null', () => {
    expect(isEmailValid(null as any)).toBe(false);
  });
});
```

Test Anatomy

Every test has 3 parts: **Arrange, Act, Assert**

```
it('should return true for valid email', () => {
  // ARRANGE - set up test data
  const input = 'test@example.com';

  // ACT - call the function being tested
  const result = isEmailValid(input);

  // ASSERT - check the result
  expect(result).toBe(true);
});
```

Common Test Patterns

Pattern 1: Testing Return Values

```
it('should return user name', () => {
  const user = { name: 'John', email: 'john@test.com' };
  expect(user.name).toBe('John');
});
```

Pattern 2: Testing Truthiness

```
it('should return true for admin', () => {
  expect(isAdmin(adminUser)).toBe(true);
  expect(isAdmin(regularUser)).toBe(false);
});
```

Pattern 3: Testing Errors

```
it('should throw error when password is too short', () => {
  expect(() => validatePassword('123'))
    .toThrow('Password must be at least 8 characters');
});
```

Pattern 4: Testing Async Functions

```
it('should return user when found', async () => {
  const user = await userService.findById('123');
  expect(user.email).toBe('test@example.com');
});
```

Pattern 5: Testing Arrays

```
it('should return list of users', () => {
  const users = getActiveUsers();
  expect(users).toHaveLength(3);
  expect(users[0].name).toBe('Alice');
});
```

Test Naming Convention

Always name tests like this:

```
should [expected behavior] when [condition]
```

Good Examples:

Test Name
`should return true when email is valid`
`should throw error when user not found`
`should create campaign when all fields provided`
`should return empty array when no results`
`should hash password when registering user`

Bad Examples:

Test Name	Problem
`test email`	Too vague
`it works`	Doesn't describe behavior
`email validation`	Not a sentence

Useful Commands

Command	What it does
`npm test`	Run all tests
`npm test -- auth`	Run tests matching "auth"
`npm test -- --watch`	Re-run tests on file changes
`npm run test:coverage`	Show coverage report
`npm test -- path/to/file.test.ts`	Run specific test file

Understanding Test Output

Passing Tests:

```
✓ src/shared/utils/__tests__/email.utils.test.ts (3 tests)
  ✓ isEmailValid
    ✓ should return true for valid email
    ✓ should return false for invalid email
    ✓ should return false for empty string

Test Files  1 passed (1)
Tests       3 passed (3)
```

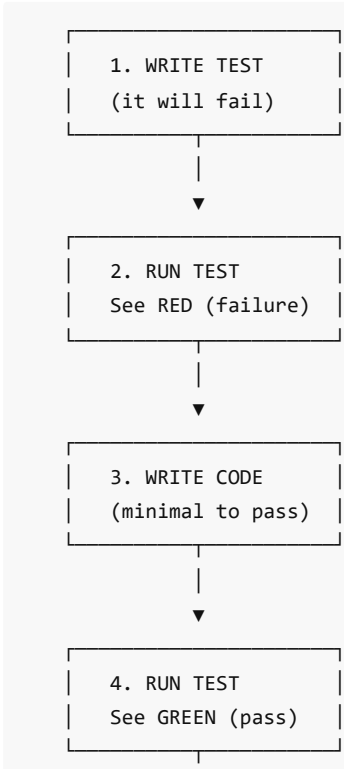
Failing Tests:

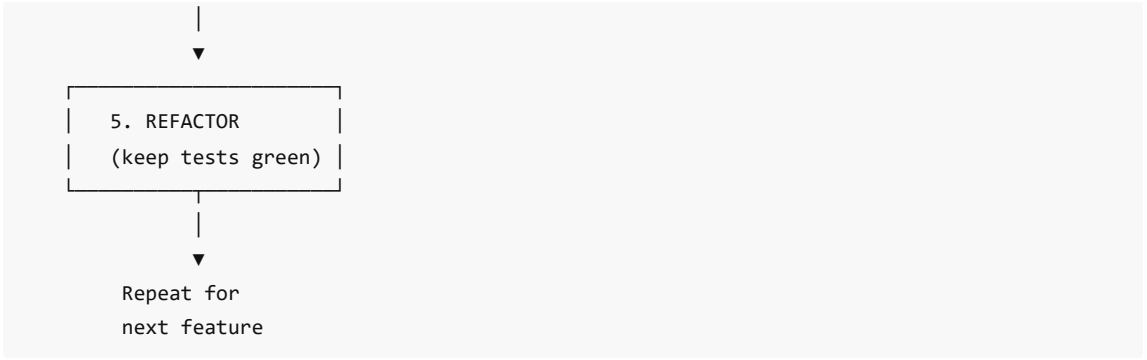
```
✗ src/shared/utils/__tests__/email.utils.test.ts (3 tests)
  ✗ isEmailValid
    ✗ should return true for valid email

      Expected: true
      Received: false

Test Files  1 failed (1)
Tests       1 failed | 2 passed (3)
```

The TDD Cycle - Visual Summary





Common Matchers Reference

Matcher	Use Case	Example
`toBe(value)`	Exact equality	`expect(1 + 1).toBe(2)`
`toEqual(obj)`	Object equality	`expect(user).toEqual({ name: 'John' })`
`toBeTruthy()`	Truthy check	`expect(user).toBeTruthy()`
`toBeFalsy()`	Falsy check	`expect(null).toBeFalsy()`
`toBeNull()`	Null check	`expect(result).toBeNull()`
`toBeUndefined()`	Undefined check	`expect(value).toBeUndefined()`
`toContain(item)`	Array contains	`expect([1,2,3]).toContain(2)`
`toHaveLength(n)`	Array/string length	`expect('hello').toHaveLength(5)`
`toThrow(msg)`	Error thrown	`expect(() => fn()).toThrow('error')`
`toMatch(regex)`	Regex match	`expect('hello').toMatch(/ell/)`

Try It Yourself - Exercise

Exercise 1: Simple Function

Create a function `isPositive(n)` that returns `true` if number is positive.

1. Create test file: `src/shared/utils/__tests__/number.utils.test.ts`
2. Write tests first
3. Create the function
4. Make tests pass

Exercise 2: String Function

Create a function `capitalize(str)` that capitalizes first letter.

- `capitalize('hello')` → `'Hello'`
- `capitalize('')` → `''`
- `capitalize('HELLO')` → `'Hello'`

Quick Tips

1. **Start small** - Write one test at a time
2. **Be specific** - Test one thing per test
3. **Use good names** - Future you will thank you
4. **Run tests often** - Catch issues early
5. **Don't skip RED** - Always see test fail first

Getting Help

- Run `npm test -- --help` for command options
- Check existing tests in `src/domains/*/` for examples
- Read `docs/TDD_DOCUMENTATION.pdf` for advanced patterns

Remember: The goal is not 100% coverage, but confidence in your code!